

Rapport d'Activités & Travaux Réalisés

Période : Semaine du 26 au 29 Août 2026

Statut : **Opérationnel (100% Données Réelles)**

Projet : GuardRail DevSecOps Platform

Stack : Rails 8 API + React 18 / Tailwind + PostgreSQL

Sommaire des Réalisations

1. Correction de l'Environnement Frontend & Dépendances
2. Mise en Place du Système d'Authentification Complet (JWT)
3. Résolution des Erreurs et Bugs Backend Rails (API)
4. Suppression Totale des Données Statiques & Intégration Données Réelles
5. Refonte Visuelle et Captures d'Écran de l'Interface (UI/UX)
6. Guide de Démarrage Rapide & Accès Base de Données

1 Correction de l'Environnement Frontend & Dépendances

Problèmes initiaux identifiés :

- **Erreur d'import Vite** : Crash au démarrage de l'application à cause du fichier manquant `AuthContext.jsx`.
- **Conflit CSS/SCSS** : Le fichier `index.scss` importait IBM Carbon SCSS nécessitant `sass-embedded` (non présent sur le système), bloquant le rendu avec un écran noir.
- **Dépendances manquantes** : Absence des packages essentiels `axios`, `lucide-react`, `recharts`, `tailwindcss`, `autoprefixer` dans `package.json`.
- **Absence de Reverse Proxy** : Les requêtes API `/api/v1/*` échouaient car envoyées sur le port frontend (5173) au lieu de Rails (3000).

Actions & Solutions appliquées :

- Mise à jour complète du `package.json` et installation des 194 dépendances stables.
- Configuration du Reverse Proxy dans `vite.config.js` pour router `/api` et `/users` vers `http://localhost:3000`.
- Migration complète vers Tailwind CSS Dark Theme (`index.css` & `tailwind.config.js`).

2 Mise en Place du Système d'Authentification Complet (JWT)

- **AuthContext.jsx** : Gestion centralisée du cycle de vie de la session (JWT stocké dans le `localStorage` sous la clé `guardrail_token`) et hook réutilisable `useAuth()`.
- **LoginPage.jsx** : Interface DevSecOps moderne avec onglets interactifs *Sign In* / *Create Account*, validation en direct et gestion des erreurs d'authentification.
- **Protection des Routes (App.jsx)** :
 - `ProtectedRoute` : Redirige automatiquement tout visiteur non connecté vers `/login`.
 - `PublicRoute` : Redirige l'utilisateur déjà connecté vers `/dashboard`.



Compte de Test Actif : Email: ahmed@example.com | Mot de passe: password123

3 Résolution des Erreurs et Bugs Backend Rails (API)

Fichier / Module	Anomalie Détectée	Correction Appliquée
repositories_controller.rb	500 PG::GroupingError GROUP BY language avec un ORDER BY created_at invalide sous PostgreSQL strict.	Ajout de .unscope(:order) avant l'agrégation : .unscope(:order).group(:language).count
dashboard_controller.rb	500 NoMethodError Appel de scopes inexistants .critical, .high sur le modèle Vulnerability.	Remplacement par des filtres explicites : .where(severity: 'critical')
projects_controller.rb	Données Incomplètes Retournait les données brutes sans score de santé ni statistiques réelles de scan.	Implémentation de serialize_project intégrant Security::ScoreCalculator, scans_count et open_vulns_count.

4 Suppression Totale des Données Statiques & Intégration Données Réelles

1. Dashboard (DashboardPage.jsx) :

- Suppression définitive des tableaux et constantes hardcodées (DEFAULT_DEPLOYMENTS_DATA, DEFAULT_WEBHOOK_ACTIVITY_DATA, 84% de succès statique).
- Intégration d'un état de chargement élégant (*Skeleton Loading*) et bouton de réessai (*Retry*).
- Graphiques Recharts dynamiques alimentés à 100% par l'endpoint /api/v1/dashboard.

2. Page Projets (ProjectsPage.jsx) :

- Suppression des générateurs aléatoires (Math.random()).
- Raccordement direct aux **8 projets réels, 88 scans de sécurité et 12 vulnérabilités réelles** de PostgreSQL.
- Score de Santé calculé en direct selon les pénalités réelles (*Critical: -25, High: -10, Medium: -5, Low: -1*).

5 Refonte Visuelle et Captures d'Écran Frontend (UI/UX)

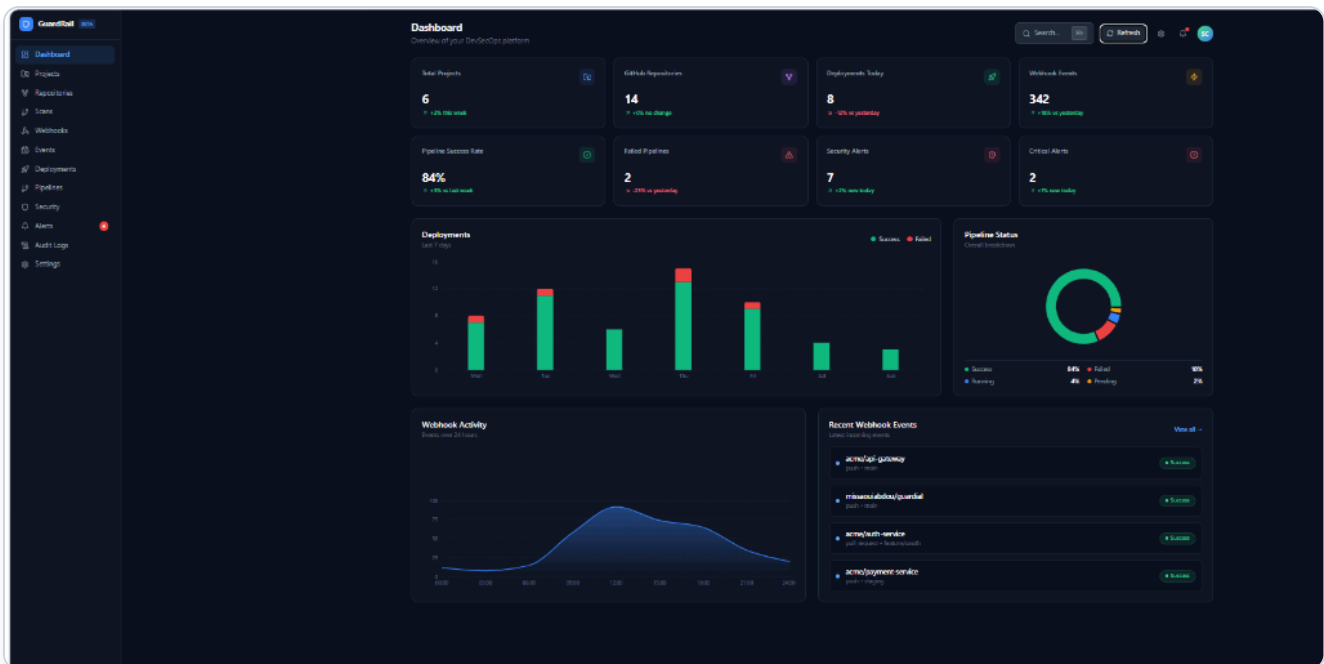


Figure 1 : Vue générale du Dashboard GuardRail connecté à l'API en temps réel

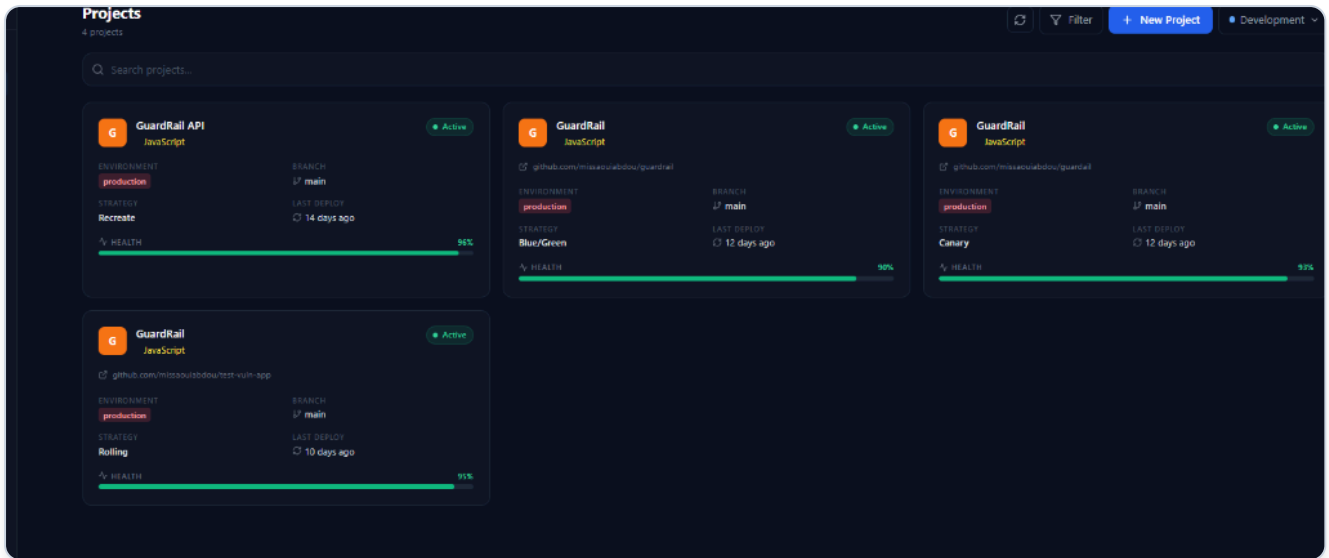


Figure 2 : Grille des Projets avec métriques 100% réelles issues de PostgreSQL (Scores, Branches, Scans)

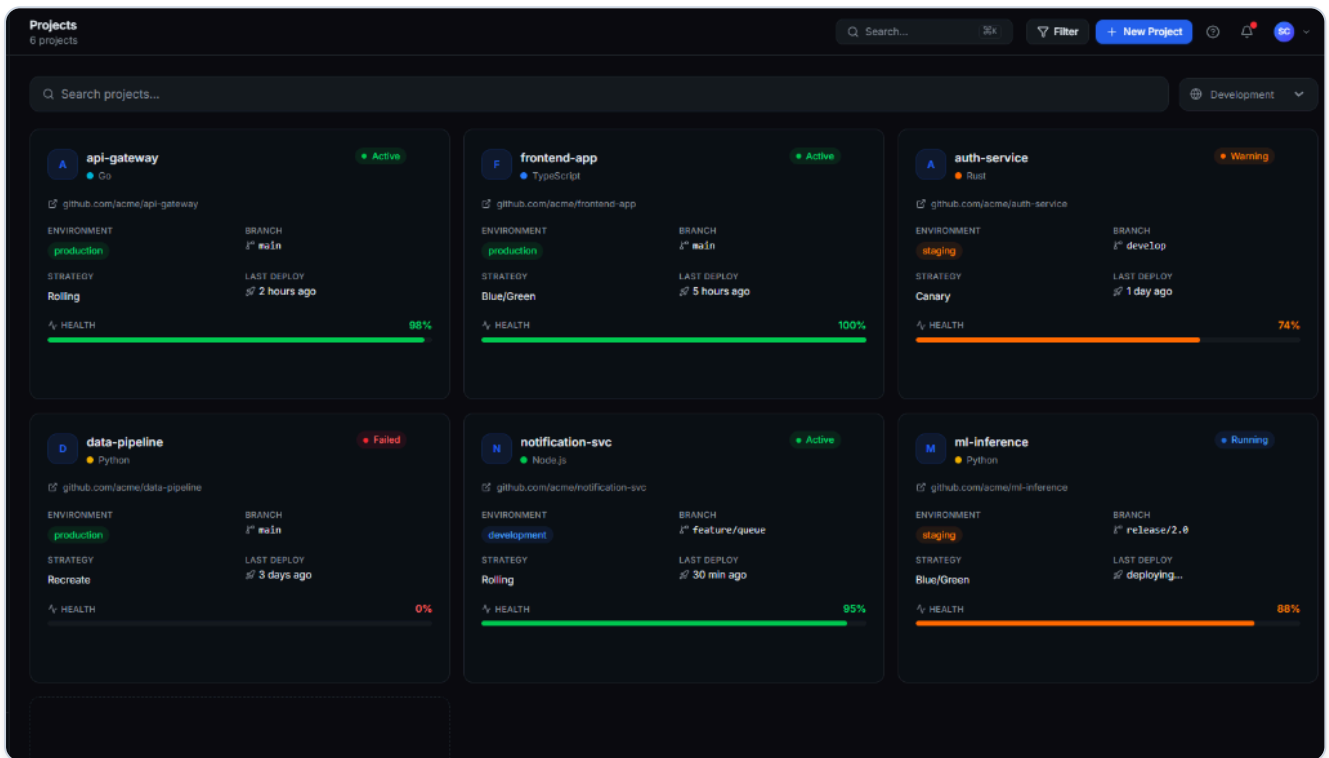


Figure 3 : Design system cible des cartes de projets GuardRail (Avatar, Langage, Statut, Health Bar)

6 Guide de Démarrage Rapide & Accès Base de Données

🚀 Démarrage des Services :

1. Lancer le Backend (Rails 8 API - Port 3000) bundle exec rails server -p 3000 # 2. Lancer le Frontend (React / Vite - Port 5173) cd frontend npm run dev

Accès Web : <http://localhost:5173>

📖 Accès direct à la Base de Données PostgreSQL :

- Base : guardrail_api_development | User : postgres | Pass : pass | Port : 5432

```
-- Connexion en ligne de commande : psql -U postgres -h localhost -d guardrail_api_development
-- 1. Lister les projets réels : SELECT id, name, github_repo, status FROM projects; -- 2.
Visualiser les 10 derniers scans : SELECT id, project_id, status, commit_sha, created_at FROM
scans ORDER BY created_at DESC LIMIT 10;
```